TECHNICAL DOCUMENTATION

#Sustenido / LiveNLoud Project Documentation

Comprehensive internal documentation for the repository currently located at `/Users/eloygomes/Documents/GitHub/new-LiveNLoud`.

Generated from the current codebase state on 2026-03-31. Frontend version: **0.62.0.0**. Chrome Extension version: **0.62.0.0**.

1. Executive Summary

This repository contains a multi-service music preparation and live performance platform. The main product identity in the UI is **#Sustenido**, while several technical and historical artifacts still use the older name **LiveNLoud**.

The platform combines:

- A React frontend for dashboard, library management, editing, presentation, tuner, metronome, and profile flows.
- A Node.js / Express API as the main authenticated application backend.
- A Python scraping service that imports song content from external music websites.
- A Python Socket.IO service used in the tuner / footswitch pipeline.
- A Chrome extension used to capture supported song pages directly from the browser.
- Docker Compose orchestration for MongoDB, Node, Python scraper, Python Socket.IO, and mailserver services.

PRIMARY USER GOAL

Manage a personal song library with per-instrument arrangements, embed videos, setlists, lyrics, chords, tabs, and live presentation tooling.

PRIMARY TECHNICAL PATTERN

Frontend calls Node API. Node delegates scraping to Python when needed. Python stores structured song data in MongoDB and mirrors shared songs into a general catalog.

2. Repository Structure

PATH	PURPOSE
`Front/`	Main React + Vite frontend application.
`Chrome Extension/`	Browser extension for quick-add workflows from supported music sites.
`Server/liveNloud/node/`	Main Node.js backend API, auth flows, CRUD, YouTube export, profile image handling.
`Server/liveNloud/python/`	Python scraper API and Mongo/general catalog persistence logic.
`Server/liveNloud/python_socketio/`	Socket.IO bridge / tuner-related processing.
`Server/liveNloud/python_tuner/`	Separate tuner service artifacts and experiments.
`Server/liveNloud/docker-compose.yml`	Local/production-like orchestration for the backend stack.
`Server/OLD/`	Legacy implementation retained for reference and migration support.
`Server/03/`	Built/static deployment artifacts and historical server output.

3. High-Level Architecture

```
Browser / Frontend / Chrome Extension
          |
          v
   Node API (Express)
          |
          +--> MongoDB
          |
          +--> Python Scraper API
          |       |
          |       +--> External sources:
          |             - Cifra Club
          |             - Ultimate Guitar
          |             - Letras / Letras.mus.br
          |
          +--> YouTube OAuth + playlist export
          |
          +--> Mailserver for email-related flows
          |
          +--> Socket.IO namespace used with Python tuner bridge
```

Core Architectural Rules

- The React app is the main operator console for authenticated users.
- Song data is user-scoped in the main app database but also mirrored into a general shared catalog.
- Scraping is isolated in Python, not in Node, which keeps source-specific parsing logic separate from business routes.
- The Chrome extension is a lightweight ingestion client and depends on the same backend APIs.
- Presentation mode is not just read-only display; it includes live performance tooling such as tooltip chords and fullscreen LIVE mode.

4. Frontend Application

4.1 Stack

- `React 18` with `react-router-dom`

- `Vite` build pipeline
- `Tailwind CSS` plus a large custom stylesheet in `Front/src/index.css`
- `Axios` for API communication
- `@tiptap/react` for in-place cifra editing
- `ChordSheetJS` for chord formatting and normalization
- `socket.io-client` for realtime integrations

4.2 Frontend Entrypoint and Routing

The main frontend entrypoint is `Front/src/main.jsx`. It creates the browser router, wraps the app in `AuthProvider` and `BluetoothProvider`, and protects internal pages using token presence in `localStorage`.

ROUTE	PAGE	NOTES
`/`	Dashboard	Main authenticated landing page.
`/newsong`	New Song	New library entry creation and scraping intake.
`/editsong/:artist/:song`	Edit Song	Edit existing entries, embeds, setlists, instrument data.
`/presentation/:artist/:song/:instrument`	Presentation	Live reading / rehearsal / stage display mode.
`/chordlibrary`	Chord Library	Reference utility page for shapes and chord lookup.
`/metronome`	Metronome	Tempo utility.
`/tuner`	Tuner	Tuning tools and related experiments.
`/userprofile/:userid`	User Profile	Account, password, user data, and profile settings.
`/login`	Login	Public auth entrypoint.
`/userregistration`	User Registration	User signup flow.
`/newpassword`	New Password	Password reset completion flow.

ROUTE	PAGE	NOTES
`/auth/spotify/callback`	Spotify Callback	OAuth completion for Spotify integrations.
`/yt/done`	YouTube Popup Done	Lightweight popup completion page for YouTube OAuth.

4.3 Important Frontend Contexts

Auth Context

`Front/src/contexts/AuthContext.jsx` is intentionally simple. It stores the access token and user email in `localStorage` and exposes `login()` and `logout()`. Routing protection is currently presence-based, not role-based.

Bluetooth Context / Footswitch Input

`Front/src/contexts/BluetoothContext.jsx` centralizes BLE and MIDI footswitch integration. It stores action mappings in `localStorage`, logs events, and dispatches navigation actions such as arrow scroll and page scroll to the current page scroll target.

> This context injects a custom `data-scroll-REMOVED_MONGO_USER` marker to identify the current scroll container. The warning seen in the browser console about `data-scroll-REMOVED_MONGO_USER` comes from this mechanism and is a known technical debt item.

4.4 Frontend API Layer

The main API client lives in `Front/src/Tools/Controllers.jsx`. It creates an Axios instance whose base URL is selected in this order:

1. `VITE_API_BASE_URL`
2. `http://localhost:3000` when running locally
3. `https://api.live.eloygomes.com` as default production fallback

Important frontend controller responsibilities include:

- User library fetch and per-song fetch.
- Song creation and exact song updates.
- Update last-played timestamps per instrument.
- User profile actions such as username changes and account deletion.

- Download of raw user JSON data.

4.5 Core Pages

Dashboard

The dashboard is the main operational screen. It supports search, list display, floating action buttons, exports, and version display. It also intentionally disables global page scroll while active.

New Song and Edit Song

These pages are the ingestion and maintenance center of the app. They maintain separate blocks for `guitar01`, `guitar02`, `bass`, `keys`, `drums`, and `voice`.

Recent behavior now documented in the current codebase:

- Letras links are voice-only sources.
- If a Letras link is pasted in another instrument, the UI redirects it to Voice and shows a warning.
- During scraping, the user receives a persistent `Carregando...` snackbar.
- After scraping, artist and song values are hydrated back into the form so save actions do not lose metadata.

Presentation

The presentation page is a performance-focused view that loads a song, selects instrument-specific content, and renders it using the custom `processSongCifra` pipeline. It supports:

- Chords, tabs, lyrics, and mixed content rendering.
- Chord tooltips with alternative fingering variation selection.
- Embedded video toolbox and draggable utility components.
- Inline cifra editing via TipTap.
- LIVE fullscreen mode with black background, white lyrics, golden chords, ESC exit, and keyboard scrolling.

The page also contains a fallback path where `songLyrics` are displayed when `songCifra` is empty, which is important for Voice/Letras content.

5. Chrome Extension

5.1 Purpose

The Chrome extension provides a lightweight “quick add” flow directly from supported source sites. It is designed to reduce friction when importing songs into the shared backend.

5.2 Main Files

- `Chrome Extension/manifest.json` — extension declaration, permissions, popup, content script.
- `Chrome Extension/page-context.js` — extracts song context from the current website.
- `Chrome Extension/popup.js` — login/session, page compatibility, save flow, notices.
- `Chrome Extension/popup.html` and `popup.css` — popup UI.

5.3 Supported Sources

SOURCE	HOST PATTERN	EXTENSION BEHAVIOR
Cifra Club	`*.cifraclub.com.br`	Extracts song, artist, key/tom, tuning, and capo.
Ultimate Guitar	`*.ultimate-guitar.com`	Extracts song, artist, key, tuning, and capo.
Letras / Letras.mus.br	`*.letras.com`, `*.letras.mus.br`	Extracts lyrics metadata and forces instrument selection to Voice.

5.4 Current UX Behavior

- All popup feedback is unified into the top notice area.
- On Letras pages, the instrument selector is locked to `voice`.
- For unsupported pages, the popup hides all normal fields and shows only the incompatible-site message.
- The popup uses `#sustenido` branding while some low-level identifiers still reference LiveNLoud.

6. Node.js Backend API

6.1 Role

The Node backend is the primary application API. It owns authentication, CRUD flows, profile image endpoints, song persistence, library retrieval, setlist handling, YouTube export, and the bridge to the Python scraper.

6.2 Implementation Note

`Server/liveNloud/node/index.js` contains a large amount of old commented code plus the active implementation later in the file. Documentation and maintenance should be based

on the active un-commented routes near the bottom of the file, not on the historical blocks at the top.

6.3 Important Route Groups

AREA	REPRESENTATIVE ENDPOINTS	PURPOSE
Authentication	`/api/auth/signup`, `/api/auth/login`, `/api/auth/refresh`, `/api/auth/updatePassword`, `/api/auth/request-password-reset`, `/api/auth/reset-password`	User auth, token lifecycle, password management.
Song CRUD	`/api/newsong`, `/api/allsongdata`, `/api/song/updateExact`, `/api/deleteonesong`	Create, fetch, patch, or delete song entries.
Library/User Data	`/api/alldata/:email`, `/api/downloadUserData/:email`, `/api/deleteAllUserSongs`, `/api/deleteUserAccount`	Full library retrieval and destructive account/data actions.
Scraping	`/api/scrape`, `/api/generalCifra`	Trigger Python scraping and resolve stored general catalog documents.
Playback / Metadata	`/api/lastPlay`, `/api/updateSetlists`	Track usage and maintain setlist metadata.
Profile	`/api/uploadProfileImage`, `/api/profileImage/:email`, `/api/updateUsername`	Profile image and identity updates.
YouTube	`/api/youtube/*`	OAuth, token refresh, and playlist export.
Health	`/health`	Basic service health check.

6.4 Data Persistence Pattern

The Node layer primarily reads and writes user-scoped song documents under the Mongo database `liveNloud_`. In addition, shared song-like artifacts are mirrored into a separate general catalog used by extension and scraping flows.

6.5 YouTube Integration

The YouTube routes under `node/youtube/` implement OAuth URL generation, state verification, token exchange, token refresh, and playlist export. The flow stores YouTube credentials per authenticated user and retries export once automatically when it detects invalid or expired credentials.

7. Python Scraper Service

7.1 Role

The Python service is an isolated HTTP API dedicated to scraping and persisting song content imported from external websites. Its main entrypoint is `Server/liveNloud/python/scrapper.py`.

7.2 Scraper Request Flow

1. Receive a POST request at `/scrape`.
2. Sanitize and normalize the incoming link.
3. Detect the source host using `source_rules.py`.
4. Reject invalid source/instrument combinations such as Letras on non-Voice fields.
5. Dispatch to the correct source-specific scraper through `scraping_service.py`.
6. Persist the normalized result to MongoDB through `storage_service.py`.
7. Return `songData` so the frontend can hydrate its form immediately.

7.3 Supported Scraping Sources

SOURCE	PYTHON SERVICE	NOTES
Cifra Club	`scraping_service_cifraclub.py`	Original/core source.
Ultimate Guitar	`scraping_service_ultimate_guitar.py`	Uses UG API first, keeps browser scraping fallback.
Letras / Letras.mus.br	`scraping_service_letrasmus.py`	Lyrics-oriented source intended for Voice instrument.

7.4 Important Current Scraper Rules

- **Letras links are Voice-only.** If the request uses another instrument, the API returns a 400 with a guidance message.
- **Lyrics-only sources can omit capo/tuning/tab fields.** Storage logic now handles missing values safely.
- **Returned payload includes `songData`.** This is required by the frontend and extension to hydrate title/artist immediately after scrape.

7.5 Storage Service Responsibilities

`storage_service.py` turns scraper output into the internal song schema. It creates a per-instrument block containing fields such as:

- `active`
- `capo`
- `tuning`
- `lastPlay`
- `songCifra`
- `songTabs`
- `songChords`
- `songLyrics`
- `progress`
- `link`

It also forwards normalized data to the general catalog through the Node API after Mongo persistence.

7.6 General Catalog Mirroring

The Python service does not only persist into the user-scoped library. It also sends a normalized version to the shared `generalCifras` path so that the platform can resolve songs by instrument, link, artist, and title across flows such as extension add and scraper retries.

8. MongoDB Data Model

8.1 User-Scoped Song Model

The dominant structure in the application stores one document per user email with a `userdata` array of songs.

```
{
  "email": "user@example.com",
  "userdata": [
    {
      "id": 1,
      "song": "Wish You Were Here",
      "artist": "Pink Floyd",
      "capo": "",
      "tom": "",
      "tuning": "",
      "progressBar": 0,
      "instruments": {
        "guitar01": true,
        "guitar02": false,
        "bass": false,
        "keys": false,
        "drums": false,
        "voice": true
      },
      "embedVideos": [],
      "setlist": [],
      "addedIn": "YYYY-MM-DD",
      "updateIn": "YYYY-MM-DD",
      "guitar01": { ...instrument block... },
      "voice": { ...instrument block... }
    }
  ]
}
```

8.2 Instrument Block Design

Each instrument stores its own content independently. This is one of the key architectural choices in the application: the same song can hold multiple arrangements and source links at once.

8.3 General Catalog Model

The shared catalog in `generalCifras` stores song-level documents that can be resolved by source link and by instrument availability. This is especially important for extension-based ingestion and for scraper-to-frontend synchronization.

9. External Integrations

INTEGRATION	WHERE USED	PURPOSE
MongoDB	Node + Python	Primary persistent storage.
Cifra Club	Python scraper + Chrome extension	Song import source.
Ultimate Guitar	Python scraper + Chrome extension	Song import source, API-first scraping path.
Letras / Letras.mus.br	Python scraper + Chrome extension	Voice/lyrics import source.
YouTube	Node backend + Dashboard export flow	Playlist export.
Spotify	Frontend callback / dashboard flow	Music service integration.
Bluetooth / Web MIDI	Frontend context	Footswitch navigation and control mapping.
Mailserver	Docker stack + Node auth flows	Email and password-reset related delivery.

10. Deployment and Runtime

10.1 Docker Compose Services

The active compose file is `Server/liveNloud/docker-compose.yml`.

SERVICE	CONTAINER ROLE	PORTS
`db`	MongoDB 7	`27017`
`mailserver`	SMTP/IMAP mail service	`25`, `465`, `587`
`node`	Main Express backend	`3000`
`python_scraper`	Python/Flask scraper service with Chromium available	`8000`
`python_socketio`	Python Socket.IO tuner/bridge service	internal

10.2 Runtime Environment Variables

Important runtime variables visible in the current repository include:

- `MONGO_URI`
- `PYTHON_API_URL`
- `NODE_ENV`
- `ACCESS_SECRET` / `REFRESH_SECRET`
- `FRONTEND_BASE_URL`
- `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`
- `CHROME_BINARY`
- `TRUSTED_HOSTS`
- `YOUTUBE_CLIENT_ID`, `YOUTUBE_CLIENT_SECRET`, `YOUTUBE_REDIRECT_URI`, `YOUTUBE_STATE_SECRET`

> Secrets are intentionally redacted in this documentation. Production deployment should always source them from environment variables or secret storage, never from committed literals.

11. Current Strengths of the Codebase

- Clear separation between scraping logic and the main Node business API.
- Per-instrument storage model is flexible and well aligned with rehearsal/performance use cases.
- Chrome extension significantly reduces friction for content ingestion.
- Presentation page now supports true stage-focused fullscreen LIVE mode.

- Ultimate Guitar and Letras integration is already wired end-to-end across backend, frontend, and extension.

12. Important Technical Debt / Risks

- **Very large backend file:** `Server/liveNloud/node/index.js` mixes active code and large commented historical blocks, which increases maintenance risk.
- **Mixed naming:** product branding is moving from LiveNLoud to #Sustenido, but many internal identifiers still use the old name.
- **LocalStorage-heavy state:** several frontend flows rely on `localStorage` as operational state, not just session state.
- **Console warning in Bluetooth context:** custom DOM attribute injection currently produces React warnings.
- **Legacy folders remain in repo:** `Server/OLD` and built deployment artifacts can confuse new maintainers.
- **Large monolithic controllers/components:** some frontend pages and the main controller layer are difficult to reason about because of broad responsibilities.

13. Recommended Refactoring Priorities

1. Split `Server/liveNloud/node/index.js` into route modules by domain: auth, songs, profile, scraping, YouTube, media.
2. Document the Mongo schema formally in a dedicated versioned spec.
3. Replace ad-hoc frontend localStorage field passing with a normalized state strategy.
4. Remove or archive obsolete deployment artifacts that are no longer part of the active system.
5. Centralize source definitions and field contracts for extension, frontend, and Python scraper to reduce drift.
6. Add explicit tests around scraper normalization and instrument persistence, especially for lyrics-only sources.

14. Operational Workflows

14.1 Add Song Through Frontend

1. User opens `New Song`.
2. User pastes source link into an instrument field.
3. Node forwards scrape request to Python.
4. Python detects source and scrapes content.

5. Python stores normalized result and returns `songData`.
6. Frontend hydrates song title, artist, and instrument content.
7. User saves final record in the app library.

14.2 Add Song Through Chrome Extension

1. User opens a supported source website.
2. Content script extracts page metadata.
3. User opens popup and reviews source, instrument, and metadata.
4. Popup calls backend save flow.
5. Backend scraper/general catalog persistence completes.
6. The song becomes available in the application library and presentation flow.

14.3 Live Performance Flow

1. User opens a presentation route for a chosen song and instrument.
2. Frontend loads the exact song entry from the backend.
3. Rendered content is normalized through the cifra parser.
4. User can show videos, edit the cifra, or switch content type.
5. User can enter LIVE mode for fullscreen stage display.
6. Keyboard arrows, page keys, BLE, or MIDI mapping can be used to navigate the content.

15. File-Level Starting Points for New Developers

AREA	START HERE
Frontend bootstrap	`Front/src/main.jsx`
Frontend API calls	`Front/src/Tools/Controllers.jsx`
Song ingestion UI	`Front/src/Pages/NewSong/` and `Front/src/Pages/EditSong/`
Presentation/live view	`Front/src/Pages/Presentation/Presentation.jsx`
Node backend	`Server/liveNloud/node/index.js`
Scraper dispatch	`Server/liveNloud/python/scrapping_service.py` conceptually; actual file is `scraping_service.py`
Source detection	`Server/liveNloud/python/source_rules.py`
Scraper persistence	`Server/liveNloud/python/storage_service.py`
Chrome extension	`Chrome Extension/popup.js` and `Chrome Extension/page-context.js`

16. Conclusion

The current repository is not a simple CRUD app. It is a hybrid music operations platform that combines song ingestion, per-instrument arrangement storage, external-source scraping, live presentation tooling, browser extension capture, and export/utility integrations. The most important architectural idea in the system is that one song can be represented differently per instrument and still remain part of a unified library and performance workflow.

The codebase is already capable of supporting a full rehearsal-to-stage flow. The next major engineering gains will come from modularization, formal schema documentation, and reducing the current amount of duplicated or historical code retained in the active repository.

End of document.